

The Model Checking Landscape of Monadic Second-Order Logic

Giovanna Kobus Conrado

September 5, 2026

Overview

What is Monadic Second-Order (MSO) Logic?

A formal language that can be used to describe sets of words/trees/graphs.

Model-checking landscape

How expressive is MSO?

For which structures is MSO logic decidable?

When decidable, what is the complexity?

This lecture: Model Checking for MSO over increasingly general structures

Finite words $\rightarrow$ Infinite words $\rightarrow$ Trees $\rightarrow$ Tree-like graphs

Overview

Why does this matter?

MSO is one of the most important **logics for graphs**, because it is:

1. *Highly expressive*: Lots of interesting graph properties are expressible on MSO (k -coloring, Hamiltonian path, feedback vertex set, vertex cover, planarity).
2. *Decidable for tree-like graphs*: We can check whether a tree-like graphs has some MSO-definable property.
3. *Admits linear verification*: We can tell whether a tree-like graph has any MSO expressible property in linear time.

MSO sits at the perfect spot in the expressivity-decidability spectrum.

But before we get there, let's start with what you know: words.

1. MSO over Finite Words

2. MSO over Infinite Words

3. MSO over Trees

4. MSO over Graphs

5. Conclusion

MSO as a logic for words

$$\varphi ::= a(x) \mid x < y \mid \neg\varphi \mid \varphi \wedge \varphi \mid \exists x \varphi \mid S(x) \mid \exists S \varphi$$

Where $a \in \Sigma$ is a letter, x, y are first-order variables (positions), and S is a second-order variable (subset of positions). We extend it with other usual operators ($\rightarrow, \neq, \forall, \dots$).

MSO as a logic for words

$$\varphi ::= a(x) \mid x < y \mid \neg\varphi \mid \varphi \wedge \varphi \mid \exists x \varphi \mid S(x) \mid \exists S \varphi$$

Where $a \in \Sigma$ is a letter, x, y are first-order variables (positions), and S is a second-order variable (subset of positions). We extend it with other usual operators ($\rightarrow, \neq, \forall, \dots$).

Example of MSO formula

$$\begin{aligned}\text{first}(x) &:= \neg\exists y. (y < x) & \text{last}(x) &:= \neg\exists y. (x < y) \\ \text{succ}(x, y) &:= (x < y) \wedge \neg\exists z. (x < z \wedge z < y)\end{aligned}$$

$$\begin{aligned}\exists S. (&\forall x. (\text{first}(x) \rightarrow S(x)) \\ &\wedge \forall x. (\text{last}(x) \rightarrow \neg S(x)) \\ &\wedge \forall x, y. (\text{succ}(x, y) \rightarrow (S(x) \neq S(y))))\end{aligned}$$

What does this do?

MSO as a logic for words

$$\varphi ::= a(x) \mid x < y \mid \neg\varphi \mid \varphi \wedge \psi \mid \exists x \varphi \mid S(x) \mid \exists S \varphi$$

Where $a \in \Sigma$ is a letter, x, y are first-order variables (positions), and S is a second-order variable (subset of positions). We extend it with other usual operators ($\rightarrow, \neq, \forall, \dots$).

Example of MSO formula

$$\begin{aligned}\text{first}(x) &:= \neg\exists y. (y < x) & \text{last}(x) &:= \neg\exists y. (x < y) \\ \text{succ}(x, y) &:= (x < y) \wedge \neg\exists z. (x < z \wedge z < y)\end{aligned}$$

$$\begin{aligned}\exists S. (&\forall x. (\text{first}(x) \rightarrow S(x)) \\ &\wedge \forall x. (\text{last}(x) \rightarrow \neg S(x)) \\ &\wedge \forall x, y. (\text{succ}(x, y) \rightarrow (S(x) \neq S(y))))\end{aligned}$$

What does this do? **Checks if the word has even length.**

How expressive is MSO (for finite words)?

Is MSO more or less expressive than regular languages?

How expressive is MSO (for finite words)?

Is MSO more or less expressive than regular languages?

They are equivalent.

Büchi-Elgot-Trakhtenbrot (BET) Theorem

A set of words is definable in MSO **if and only if** it is a regular language.

We will prove both directions.

Büchi-Elgot-Trakhtenbrot Theorem

How do we prove that every regular language is MSO-definable?

Büchi-Elgot-Trakhtenbrot Theorem

How do we prove that every regular language is MSO-definable?

We take the DFA for that language, and translate it into an MSO formula.

Büchi-Elgot-Trakhtenbrot Theorem

How do we prove that every regular language is MSO-definable?

We take the DFA for that language, and translate it into an MSO formula.

Regular Language $\rightarrow$ MSO-definable

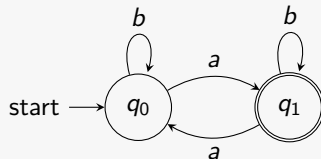
We have a deterministic finite automaton $A = (\Sigma, Q, q_{\text{ini}}, \delta, F)$, and want to construct a corresponding MSO formula φ .

Büchi-Elgot-Trakhtenbrot Theorem

Regular Language $\rightarrow$ MSO-definable

We want to check if the word is accepted by the automata:

aababb
123456



Map each position to the state I am at *before* that letter. Is this valid?

$q_0 : 1, 3, 4$ $q_1 : 2, 5, 6$

Büchi-Elgot-Trakhtenbrot Theorem

Regular Language $\rightarrow$ MSO-definable

We have a deterministic finite automaton $A = (\Sigma, Q, q_{\text{ini}}, \delta, F)$, and want to construct a corresponding MSO formula φ . On a high-level:

- ◇ We map each position in the string to an $q_i \in Q$.
- ◇ The start of the string is mapped to q_{ini} .
- ◇ The end of the string is mapped to some state preceding $q_f \in F$.
- ◇ Two adjacent positions x and y must be such that their corresponding states are connected by a transition with label $\sigma(x)$ (the letter in position x).

Büchi-Elgot-Trakhtenbrot Theorem

Regular Language $\rightarrow$ MSO-definable

Let us state this in MSO.

$$\begin{aligned} \varphi := \exists S_{q_1}, S_{q_2}, \dots \quad & \forall x. \left(\bigvee_{q_i \in Q} S_{q_i}(x) \wedge \bigwedge_{q_i, q_j \in Q} S_{q_i}(x) \rightarrow \neg S_{q_j}(x) \right) \\ & \wedge \forall x. (\text{first}(x) \rightarrow S_{q_{\text{ini}}}(x)) \\ & \wedge \forall x. \left(\text{last}(x) \rightarrow \bigvee_{(q_i, \sigma, q_f) \in \delta, q_f \in F} S_{q_i}(x) \wedge \sigma(x) \right) \\ & \wedge \forall x, y. \left(\text{succ}(x, y) \rightarrow \left(\bigvee_{(q_i, \sigma, q_j) \in \delta} S_{q_i}(x) \wedge \sigma(x) \wedge S_{q_j}(y) \right) \right) \end{aligned}$$

We define the sets of positions mapped to each state.

Büchi-Elgot-Trakhtenbrot Theorem

Regular Language $\rightarrow$ MSO-definable

Let us state this in MSO.

$$\begin{aligned}\varphi := \exists S_{q_1}, S_{q_2}, \dots \quad & \forall x. \left(\bigvee_{q_i \in Q} S_{q_i}(x) \wedge \bigwedge_{q_i, q_j \in Q} S_{q_i}(x) \rightarrow \neg S_{q_j}(x) \right) \\ & \wedge \forall x. (\text{first}(x) \rightarrow S_{q_{\text{ini}}}(x)) \\ & \wedge \forall x. \left(\text{last}(x) \rightarrow \bigvee_{(q_i, \sigma, q_f) \in \delta, q_f \in F} S_{q_i}(x) \wedge \sigma(x) \right) \\ & \wedge \forall x, y. \left(\text{succ}(x, y) \rightarrow \left(\bigvee_{(q_i, \sigma, q_j) \in \delta} S_{q_i}(x) \wedge \sigma(x) \wedge S_{q_j}(y) \right) \right)\end{aligned}$$

Each position is mapped to exactly one state.

Büchi-Elgot-Trakhtenbrot Theorem

Regular Language $\rightarrow$ MSO-definable

Let us state this in MSO.

$$\begin{aligned}\varphi := \exists S_{q_1}, S_{q_2}, \dots \quad & \forall x. \left(\bigvee_{q_i \in Q} S_{q_i}(x) \wedge \bigwedge_{q_i, q_j \in Q} S_{q_i}(x) \rightarrow \neg S_{q_j}(x) \right) \\ & \wedge \forall x. (\text{first}(x) \rightarrow S_{q_{\text{ini}}}(x)) \\ & \wedge \forall x. \left(\text{last}(x) \rightarrow \bigvee_{(q_i, \sigma, q_f) \in \delta, q_f \in F} S_{q_i}(x) \wedge \sigma(x) \right) \\ & \wedge \forall x, y. \left(\text{succ}(x, y) \rightarrow \left(\bigvee_{(q_i, \sigma, q_j) \in \delta} S_{q_i}(x) \wedge \sigma(x) \wedge S_{q_j}(y) \right) \right)\end{aligned}$$

The first position of the string is mapped to q_{ini} .

Büchi-Elgot-Trakhtenbrot Theorem

Regular Language $\rightarrow$ MSO-definable

Let us state this in MSO.

$$\begin{aligned}\varphi := \exists S_{q_1}, S_{q_2}, \dots \quad & \forall x. \left(\bigvee_{q_i \in Q} S_{q_i}(x) \wedge \bigwedge_{q_i, q_j \in Q} S_{q_i}(x) \rightarrow \neg S_{q_j}(x) \right) \\ & \wedge \forall x. (\text{first}(x) \rightarrow S_{q_{\text{ini}}}(x)) \\ & \wedge \forall x. \left(\text{last}(x) \rightarrow \bigvee_{(q_i, \sigma, q_f) \in \delta, q_f \in F} S_{q_i}(x) \wedge \sigma(x) \right) \\ & \wedge \forall x, y. \left(\text{succ}(x, y) \rightarrow \left(\bigvee_{(q_i, \sigma, q_j) \in \delta} S_{q_i}(x) \wedge \sigma(x) \wedge S_{q_j}(y) \right) \right)\end{aligned}$$

Last position is mapped to a state that points to $q_f \in F$ "using" the last letter.

Büchi-Elgot-Trakhtenbrot Theorem

Regular Language $\rightarrow$ MSO-definable

Let us state this in MSO.

$$\begin{aligned}\varphi := \exists S_{q_1}, S_{q_2}, \dots \quad & \forall x. \left(\bigvee_{q_i \in Q} S_{q_i}(x) \wedge \bigwedge_{q_i, q_j \in Q} S_{q_i}(x) \rightarrow \neg S_{q_j}(x) \right) \\ & \wedge \forall x. (\text{first}(x) \rightarrow S_{q_{\text{ini}}}(x)) \\ & \wedge \forall x. \left(\text{last}(x) \rightarrow \bigvee_{(q_i, \sigma, q_f) \in \delta, q_f \in F} S_{q_i}(x) \wedge \sigma(x) \right) \\ & \wedge \forall x, y. \left(\text{succ}(x, y) \rightarrow \left(\bigvee_{(q_i, \sigma, q_j) \in \delta} S_{q_i}(x) \wedge \sigma(x) \wedge S_{q_j}(y) \right) \right)\end{aligned}$$

The states of adjacent positions x and y are connected by a transition with label $\sigma(x)$.

Büchi-Elgot-Trakhtenbrot Theorem

Regular Language $\rightarrow$ MSO-definable

Let us state this in MSO.

$$\begin{aligned}\varphi := \exists S_{q_1}, S_{q_2}, \dots \quad & \forall x. \left(\bigvee_{q_i \in Q} S_{q_i}(x) \wedge \bigwedge_{q_i, q_j \in Q} S_{q_i}(x) \rightarrow \neg S_{q_j}(x) \right) \\ & \wedge \forall x. (\text{first}(x) \rightarrow S_{q_{\text{ini}}}(x)) \\ & \wedge \forall x. \left(\text{last}(x) \rightarrow \bigvee_{(q_i, \sigma, q_f) \in \delta, q_f \in F} S_{q_i}(x) \wedge \sigma(x) \right) \\ & \wedge \forall x, y. \left(\text{succ}(x, y) \rightarrow \left(\bigvee_{(q_i, \sigma, q_j) \in \delta} S_{q_i}(x) \wedge \sigma(x) \wedge S_{q_j}(y) \right) \right)\end{aligned}$$

So any regular language can be written to MSO!

Büchi-Elgot-Trakhtenbrot Theorem

Let us prove the other direction.

MSO-definable $\rightarrow$ Regular Language

Let us do the reverse: transform a formula into a DFA.

1. Define an automaton that identifies each atomic formula (e.g, $a(x)$).
2. Recursively define how to merge automata (e.g, $\varphi \wedge \varphi$).

Büchi-Elgot-Trakhtenbrot Theorem

Problem

Things like $\exists x$ and $\exists S$ create free variables.

Take $\varphi ::= \exists S, T. (\exists x, y. (\gamma))$.

What does the automaton for γ look like? How does it know what is x and y ?

Büchi-Elgot-Trakhtenbrot Theorem

Problem

Things like $\exists x$ and $\exists S$ create free variables.

Take $\varphi ::= \exists S, T. (\exists x, y. (\gamma))$.

What does the automaton for γ look like? How does it know what is x and y ?

Solution

Annotate words with information on free variables.

- ◇ **words** $\rightarrow (V, W)$ -**words**.
 - ◇ V is the set of free variables (e.g., x, y).
 - ◇ W is the set of free sets of variables (e.g., S, T).

Büchi-Elgot-Trakhtenbrot Theorem

(V, W) -words

- ◇ V is the set of free variables (e.g., x, y).
- ◇ W is the set of free sets of variables (e.g., S, T).

Before: position i is mapped to character a .

Now: position i is mapped to:

- ◇ character a
- ◇ variables $x \in V$ that are assigned to this position
- ◇ sets $S \in W$ that contain this position

Büchi-Elgot-Trakhtenbrot Theorem

Example of (V, W) -word

$\varphi ::= \exists S, T. (\exists x, y. (\gamma))$, for some subformula γ .

What types of words should the automaton of γ recognize?

Büchi-Elgot-Trakhtenbrot Theorem

Example of (V, W) -word

$\varphi ::= \exists S, T. (\exists x, y. (\gamma))$, for some subformula γ .

What types of words should the automaton of γ recognize?

Answer: $(\{x, y\}, \{S, T\})$ -words.

abba	a	b	b	a
	x	y		
	S	S	S	
	T			T

This indicates an instance where $x = 1, y = 2, S = \{1, 2, 3\}$, and $T = \{1, 4\}$. It is easy to write an automaton that accepts all valid $(\{x, y\}, \{S, T\})$ -words. So our automata can assume that only valid words will be given as an input.

Büchi-Elgot-Trakhtenbrot Theorem

MSO-definable $\rightarrow$ Regular Language (Atomic automaton over (V, W) -words)

Now we need to write automata over (V, W) -words. What do they look like?

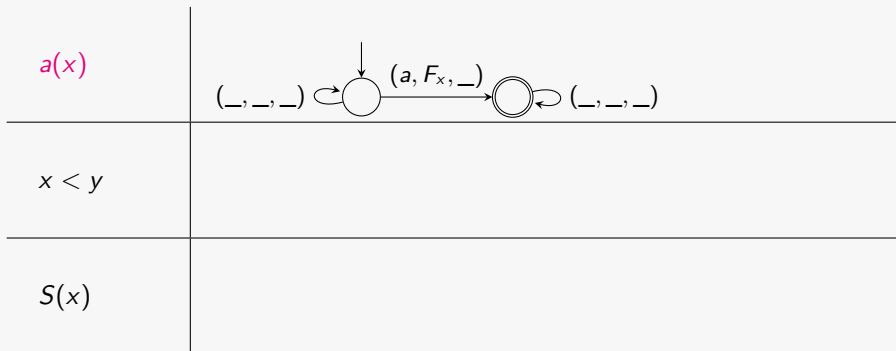
$a(x)$	
$x < y$	
$S(x)$	

Where F_x is any $F \subseteq V$ s.t. $x \in F$ and F_S is any $F \subseteq W$ s.t. $S \in F$.

Büchi-Elgot-Trakhtenbrot Theorem

MSO-definable $\rightarrow$ Regular Language (Atomic automaton over (V, W) -words)

Now we need to write automata over (V, W) -words. What do they look like?

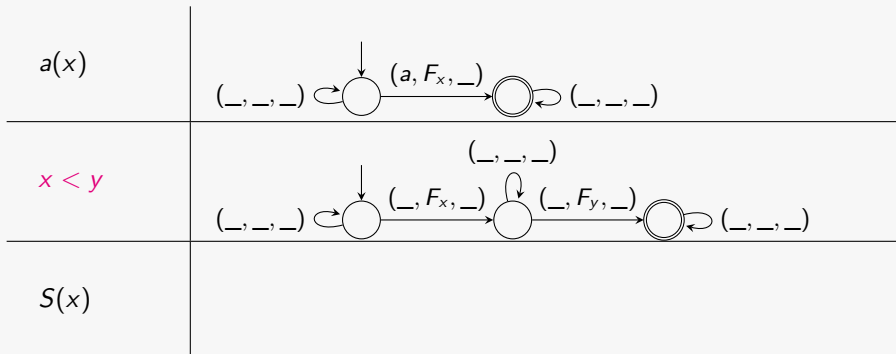


Where F_x is any $F \subseteq V$ s.t. $x \in F$ and F_S is any $F \subseteq W$ s.t. $S \in F$.

Büchi-Elgot-Trakhtenbrot Theorem

MSO-definable $\rightarrow$ Regular Language (Atomic automaton over (V, W) -words)

Now we need to write automata over (V, W) -words. What do they look like?

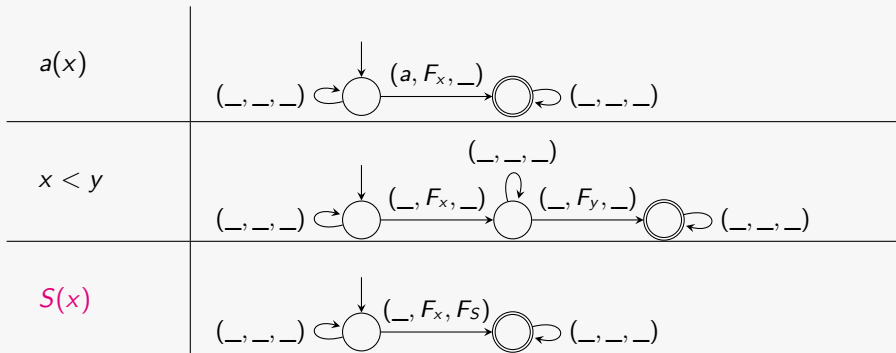


Where F_x is any $F \subseteq V$ s.t. $x \in F$ and F_y is any $F \subseteq W$ s.t. $S \in F$.

Büchi-Elgot-Trakhtenbrot Theorem

MSO-definable $\rightarrow$ Regular Language (Atomic automaton over (V, W) -words)

Now we need to write automata over (V, W) -words. What do they look like?



Where F_x is any $F \subseteq V$ s.t. $x \in F$ and F_S is any $F \subseteq W$ s.t. $S \in F$.

Büchi-Elgot-Trakhtenbrot Theorem

MSO-definable $\rightarrow$ Regular Language (Automata composition over (V, W) -words)

Let us use some results from regular languages instead of trying to be smart.

$\neg\varphi$	Regular languages are closed under complement
$\varphi \wedge \varphi$	Regular languages are closed under language intersection.
$\exists x.\varphi$	Must accept all (V, W) -words accepted by φ after “forgetting” the x component $((V - \{x\}, W)$ -words). Regular languages are closed under projection.
$\exists S.\varphi$	Same procedure as above, but with $(V, W - \{S\})$ -words.

Everything else can be expressed through these atoms and operations $(\rightarrow, \neq, \forall, \dots)$.

MSO-automata equivalencies

We will omit details in the following sections, but this is the general structure these proofs follows:

Identifiable by automata $\rightarrow$ MSO-definable

We write that automata as an MSO formula.

MSO-definable $\rightarrow$ Identifiable by automata

- ◇ First translate atoms (using some variant of (V, W) -words).
- ◇ Use automata properties to prove negation ($\neg$), intersection ($\wedge$) and exists ($\exists$).

Note that, for this last point, we use the fact that the automata is closed under complement, intersection and projection.

MSO-automata equivalencies

We will omit details in the following sections, but this is the general structure these proofs follows:

Identifiable by automata $\rightarrow$ MSO-definable

We write that automata as an MSO formula.

MSO-definable $\rightarrow$ Identifiable by automata

- ◇ First translate atoms (using some variant of (V, W) -words).
- ◇ Use automata properties to prove negation ($\neg$), intersection ($\wedge$) and exists ($\exists$).

Note that, for this last point, we use the fact that the automata is closed under complement, intersection and projection.

Note: If satisfiability, validity, emptiness etc. is decidable this type of automata, it is also decidable for any MSO formula!

Complexity of MSO Model Checking

Good news

For a **fixed** MSO formula φ , model checking a word w takes $O(|w|)$ time.

Bad news: Meyer-Stockmeyer

There is no translation of MSO formulas of size n to automata whose size is bounded by any fixed tower of exponentials:

$$2^{\dots^{2^n}} \} k \quad k \text{ constant, independent of } n$$

This tower of powers come from nested $\exists S$, so avoid those if you can.
In practice, it is not that bad (there are even decent/practical tools!).

Since no one is ever satisfied with a decidable logic, *let's expand our model*.

1. MSO over Finite Words
- 2. MSO over Infinite Words**
3. MSO over Trees
4. MSO over Graphs
5. Conclusion

What are infinite words?

Infinite words

An infinite word over Σ is $\alpha : \mathbb{N} \rightarrow \Sigma$.

The syntax of MSO extends naturally.

$$\varphi ::= a(x) \mid x < y \mid \neg\varphi \mid \varphi \wedge \varphi \mid \exists x \varphi \mid S(x) \mid \exists S \varphi$$

Where $a \in \Sigma$ is a letter, x, y are first-order variables (positions), and S is a second-order variable (subset of positions). We extend it with other usual operators ($\rightarrow, \neq, \forall, \dots$).

What are infinite words?

Infinite words

An infinite word over Σ is $\alpha : \mathbb{N} \rightarrow \Sigma$.

The syntax of MSO extends naturally.

$$\varphi ::= a(x) \mid x < y \mid \neg\varphi \mid \varphi \wedge \varphi \mid \exists x \varphi \mid S(x) \mid \exists S \varphi$$

Where $a \in \Sigma$ is a letter, x, y are first-order variables (positions), and S is a second-order variable (subset of positions). We extend it with other usual operators ($\rightarrow, \neq, \forall, \dots$).

Example of MSO formula

$$\neg\exists x.(a(x)) \vee \exists x.(a(x) \wedge \forall y.((y > x) \implies \neg a(y)))$$

What does this do?

What are infinite words?

Infinite words

An infinite word over Σ is $\alpha : \mathbb{N} \rightarrow \Sigma$.

The syntax of MSO extends naturally.

$$\varphi ::= a(x) \mid x < y \mid \neg\varphi \mid \varphi \wedge \varphi \mid \exists x \varphi \mid S(x) \mid \exists S \varphi$$

Where $a \in \Sigma$ is a letter, x, y are first-order variables (positions), and S is a second-order variable (subset of positions). We extend it with other usual operators ($\rightarrow, \neq, \forall, \dots$).

Example of MSO formula

$$\neg \exists x. (a(x)) \vee \exists x. (a(x) \wedge \forall y. ((y > x) \implies \neg a(y)))$$

What does this do? **Checks if the word has finitely many “a”s.**

From finite to infinite words

We now move from finite words to infinite words. Recall that the key ingredients of our proof of decidability are:

1. A class of automata we can use to model MSO
2. Automata properties sufficient to model negation ($\neg$), intersection ($\wedge$) and exists ($\exists$)

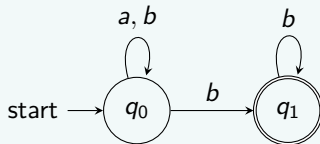
For finite words, DFA worked perfectly. What should we use for infinite words?

Büchi automata

The “DFA” of infinite words is Büchi automata.

Nondeterministic Büchi automata

A word is accepted by a NBA if it has a run the visits a final state infinitely many times.



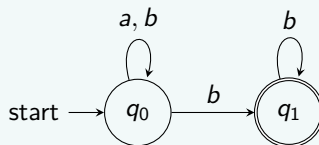
What does this do?

Büchi automata

The “DFA” of infinite words is Büchi automata.

Nondeterministic Büchi automata

A word is accepted by a NBA if it has a run that visits a final state infinitely many times.



What does this do? **Checks if the word has finitely many “a”s.**

Properties of Nondeterministic Büchi automata

As we saw in the finite setting, we need to model negation ($\neg$), intersection ($\wedge$) and exists ($\exists$).

Do NBA make the cut? Let's try to exploit automata properties once again.

Properties of Nondeterministic Büchi automata

As we saw in the finite setting, we need to model negation ($\neg$), intersection ($\wedge$) and exists ($\exists$).

Do NBA make the cut? Let's try to exploit automata properties once again.

$\neg\varphi$	ω -regular languages are closed under complement
$\varphi \wedge \varphi$	ω -regular languages are closed under language intersection.
$\exists x.\varphi$	ω -regular are closed under projection.
$\exists S.\varphi$	Same as above.

Looks good!

Decidability of MSO on infinite words

Büchi's Theorem (1962)

A language $L \subseteq \Sigma^\omega$ is definable in MSO if and only if L is ω -regular.

The proof reuses the exact same structure as BET, but for a different automata class.

Sanity check

We have been using nondeterministic Büchi automata.

Could we have used deterministic Büchi automata instead?

Sanity check

We have been using nondeterministic Büchi automata.

Could we have used deterministic Büchi automata instead?

No, because it is not closed under complement!

Decidability of MSO on infinite words

MSO model checking: decidability so far

Finite words decidable (BET)

Infinite words decidable (Büchi's Theorem)

Decidability of MSO on infinite words

MSO model checking: decidability so far

Finite words	decidable (BET)
Infinite words	decidable (Büchi's Theorem)

MSO characterization

Finite words	$\leftrightarrow$ regular languages
Infinite words	$\leftrightarrow \omega$ -regular languages

Decidability of MSO on infinite words

MSO model checking: decidability so far

Finite words	decidable (BET)
Infinite words	decidable (Büchi's Theorem)

MSO characterization

Finite words	$\leftrightarrow$ regular languages
Infinite words	$\leftrightarrow \omega$ -regular languages

Both settings share the same non-elementary complexity in the formula size.

Still too easy! Let's move to something even more complex.

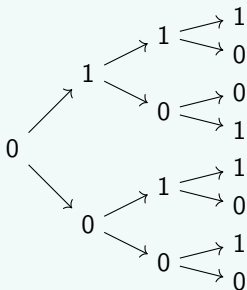
1. MSO over Finite Words
2. MSO over Infinite Words
- 3. MSO over Trees**
4. MSO over Graphs
5. Conclusion

MSO as a logic for trees

MSO on (infinite) words $\rightarrow$ S1S (one successor)

$$1 \rightarrow 0 \rightarrow 0 \rightarrow 1$$

MSO on (infinite) trees $\rightarrow$ S2S (two successors)



MSO as a logic for trees

We start with finite trees.

$$\varphi ::= a(x) \mid \text{succ}_L(x, y) \mid \text{succ}_R(x, y) \mid S(x) \mid \neg\varphi \mid \varphi \wedge \varphi \mid \exists x \varphi \mid \exists S \varphi$$

Where x, y are first-order variables (vertices), and S is a second-order variable (vertices). We extend it with other usual operators ($\rightarrow, \neq, \forall, \dots$).

Although we have binary predicates (edges), quantification over sets is restricted to monadic predicates only.

MSO as a logic for trees

Example of MSO formula

$$\text{hc}_L(x) := \exists y. (\text{succ}_L(x, y)) \quad \text{hc}_R(x) := \exists y. (\text{succ}_R(x, y))$$

$$\text{root}(x) := \neg \exists y. (\text{succ}_L(y, x) \vee \text{succ}_R(y, x))$$

$$\exists S. (\forall x. (\text{root}(x) \implies S(x)$$

$$\wedge (\neg \text{hc}_L(x) \wedge \neg \text{hc}_R(x)) \implies S(x)$$

$$\wedge (\text{hc}_L(x) \wedge \neg \text{hc}_R(x)) \implies (\exists y. (\text{succ}_L(x, y) \wedge S(x) = S(y)))$$

$$\wedge (\neg \text{hc}_L(x) \wedge \text{hc}_R(x)) \implies (\exists y. (\text{succ}_R(x, y) \wedge S(x) = S(y)))$$

$$\wedge (\text{hc}_L(x) \wedge \text{hc}_R(x)) \implies (\exists y_L, y_R. (\text{succ}_L(x, y_L) \wedge \text{succ}_R(x, y_R) \\ \wedge S(x) = (S(y_L) \neq S(y_R)))))$$

What does this do?

MSO as a logic for trees

Example of MSO formula

$$\text{hc}_L(x) := \exists y. (\text{succ}_L(x, y)) \quad \text{hc}_R(x) := \exists y. (\text{succ}_R(x, y))$$

$$\text{root}(x) := \neg \exists y. (\text{succ}_L(y, x) \vee \text{succ}_R(y, x))$$

$$\exists S. (\forall x. (\text{root}(x) \implies S(x)$$

$$\wedge (\neg \text{hc}_L(x) \wedge \neg \text{hc}_R(x)) \implies S(x)$$

$$\wedge (\text{hc}_L(x) \wedge \neg \text{hc}_R(x)) \implies (\exists y. (\text{succ}_L(x, y) \wedge S(x) = S(y)))$$

$$\wedge (\neg \text{hc}_L(x) \wedge \text{hc}_R(x)) \implies (\exists y. (\text{succ}_R(x, y) \wedge S(x) = S(y)))$$

$$\wedge (\text{hc}_L(x) \wedge \text{hc}_R(x)) \implies (\exists y_L, y_R. (\text{succ}_L(x, y_L) \wedge \text{succ}_R(x, y_R) \\ \wedge S(x) = (S(y_L) \neq S(y_R)))))$$

What does this do? **Checks if the tree has an odd number of leaves.**

Why can't I just define the set of trees and see if it is even?

From words to trees

Recall that the key ingredients of our proof of decidability are:

1. A class of automata we can use to model MSO
2. Automata properties sufficient to model negation ($\neg$), intersection ($\wedge$) and exists ($\exists$)

From words to trees

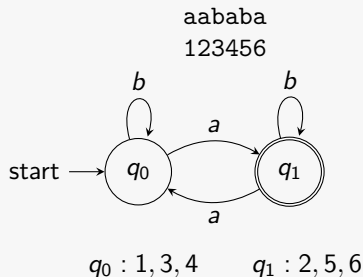
Recall that the key ingredients of our proof of decidability are:

1. A class of automata we can use to model MSO
2. Automata properties sufficient to model negation ($\neg$), intersection ($\wedge$) and exists ($\exists$)

We need an automaton for trees??? Let's recall how automaton on strings work and generalize this idea.

String Automata

A run of a string automaton assigns a state to each position. The state of a position depends only on the state of the previous position.



To find the state of position 6 we need:

- ◇ The state of the previous position (5, q_1)
- ◇ The character at position 6 (a)

This nets us the state q_1 .

Tree Automata

A run of a tree automaton assigns a state to each vertex. The state of a vertex depends only on the state of its children.

We call this a *bottom-up tree automata*.

Important differences

DFA

Position $\rightarrow$ state
State depends (1) previous position
Left-to-right processing
Look nice :)

Tree Automata

Vertex $\rightarrow$ state
State depends on (0, 1 or 2) children
Bottom-up processing
Just some rules :(

Tree automata

The definition looks very similar to DFA.

Bottom-up finite tree automaton (FTA)

A **bottom-up tree automaton** over ranked alphabet Σ is a tuple $\mathcal{A} = (Q, \Sigma, \Delta, F)$ where:

- ◇ Q is a finite set of states
- ◇ Σ is a **ranked alphabet** (each symbol $a \in \Sigma$ has arity $n \geq 0$)
- ◇ $\Delta \subseteq \bigcup_{a \in \Sigma} Q^{\text{arity}(a)} \times \{a\} \times Q$ is the transition relation
- ◇ $F \subseteq Q$ is the set of accepting states

We write $(q_1, \dots, q_n) \xrightarrow{a} q$ for $(q_1, \dots, q_n, a, q) \in \Delta$.

A run assigns states to nodes bottom-up. A tree is accepted if the root reaches a state in F .

Intuitively, the arity encodes the fact that there are **symbols for leaves**, and there are **symbols for inner nodes**. Let's look an example.

Tree automata

Example of Tree Automaton

Let $Q = \{\text{even}, \text{odd}\}$, $\Sigma = \{a^{(2)}, b^{(1)}, c^{(0)}\}$, $F = \{\text{odd}\}$

$$\begin{aligned}\Delta : \quad & () \xrightarrow{c} \text{odd}, (\text{even}) \xrightarrow{b} \text{even}, (\text{odd}) \xrightarrow{b} \text{odd} \\ & (\text{even}, \text{even}) \xrightarrow{a} \text{even} \\ & (\text{even}, \text{odd}) \xrightarrow{a} \text{odd} \\ & (\text{odd}, \text{even}) \xrightarrow{a} \text{odd} \\ & (\text{odd}, \text{odd}) \xrightarrow{a} \text{even}\end{aligned}$$

What does this do?

Tree automata

Example of Tree Automaton

Let $Q = \{\text{even}, \text{odd}\}$, $\Sigma = \{a^{(2)}, b^{(1)}, c^{(0)}\}$, $F = \{\text{odd}\}$

$$\begin{aligned}\Delta : \quad & () \xrightarrow{c} \text{odd}, (\text{even}) \xrightarrow{b} \text{even}, (\text{odd}) \xrightarrow{b} \text{odd} \\ & (\text{even}, \text{even}) \xrightarrow{a} \text{even} \\ & (\text{even}, \text{odd}) \xrightarrow{a} \text{odd} \\ & (\text{odd}, \text{even}) \xrightarrow{a} \text{odd} \\ & (\text{odd}, \text{odd}) \xrightarrow{a} \text{even}\end{aligned}$$

What does this do? **Accepts binary trees that have an odd number of leaves.**

Properties of tree automata

- ◇ Deterministic bottom-up tree automata are as expressive as nondeterministic bottom-up tree automata.
- ◇ Closed under complement, intersection, and projection.

As you may have guessed, the tree language expressible in S2S (MSO for trees) are those recognizable by tree automata.

Both are decidable.

What about infinite trees?

Bottom-up infinite tree automaton (ITA)

This is the Büchi automata of tree automata.

Acceptance is defined as a condition on all infinite paths on the tree.

- ◇ There are many variants (Muller, Rabin, Streett, parity, Büchi).
- ◇ Büchi is weaker, rest are equivalent.
- ◇ Nondeterministic variants are stronger than deterministic variants.
- ◇ Closed under complement, intersection, and projection.

They are, indeed, equivalent on S2S for infinite trees. But is this decidable?

Is this decidable?

Michael Rabin (1969)

Emptiness on infinite tree automata is decidable and infinite tree automata are equivalent to S2S.

Therefore, S2S is decidable.

Is this decidable?

Michael Rabin (1969)

Emptiness on infinite tree automata is decidable and infinite tree automata are equivalent to S2S.

Therefore, S2S is decidable.

This was shocking.

It is still one of the most expressive natural decidable theories known, and a lot of decidable theories interpret to this.

Decidability of MSO on infinite words

MSO model checking: decidability so far

Finite words	decidable (BET)
Infinite words	decidable (Büchi's Theorem)
Finite trees	decidable
Infinite trees	decidable (Rabin)

Decidability of MSO on infinite words

MSO model checking: decidability so far

Finite words	decidable (BET)
Infinite words	decidable (Büchi's Theorem)
Finite trees	decidable
Infinite trees	decidable (Rabin)

MSO characterization

Finite words	$\leftrightarrow$ regular languages
Infinite words	$\leftrightarrow \omega$ -regular languages
Finite trees	$\leftrightarrow$ regular tree languages
Infinite trees	$\leftrightarrow \omega$ -regular tree languages

Decidability of MSO on infinite words

MSO model checking: decidability so far

Finite words	decidable (BET)
Infinite words	decidable (Büchi's Theorem)
Finite trees	decidable
Infinite trees	decidable (Rabin)

MSO characterization

Finite words	$\leftrightarrow$ regular languages
Infinite words	$\leftrightarrow \omega$ -regular languages
Finite trees	$\leftrightarrow$ regular tree languages
Infinite trees	$\leftrightarrow \omega$ -regular tree languages

Still non-elementary complexity in the formula size and but admit linear time testing for a fixed formula.

Let's use this result to go even further.

1. MSO over Finite Words
2. MSO over Infinite Words
3. MSO over Trees
- 4. MSO over Graphs**
5. Conclusion

MSO as a logic for graphs

MSO extends naturally to (undirected) graphs.

$$\varphi ::= \text{edge}(x, y) \mid S(x) \mid \neg\varphi \mid \varphi \wedge \varphi \mid \exists x \varphi \mid \exists S \varphi$$

Where x, y are first-order variables (vertices), and S is a second-order variable (subset of vertices). We extend it with other usual operators ($\rightarrow, \neq, \forall, \dots$).

Although we have binary predicates (edges), quantification over sets is restricted to monadic predicates only.

MSO as a logic for graphs

Example of MSO formula

$$\begin{aligned} \exists S_1 \exists S_2 \exists S_3 (&\forall x (S_1(x) \vee S_2(x) \vee S_3(x)) \\ &\wedge \forall x \neg (S_1(x) \wedge S_2(x)) \wedge \neg (S_1(x) \wedge S_3(x)) \wedge \neg (S_2(x) \wedge S_3(x)) \\ &\wedge \forall x, y (\text{edge}(x, y) \rightarrow \neg (S_1(x) \wedge S_1(y))) \\ &\wedge \forall x, y (\text{edge}(x, y) \rightarrow \neg (S_2(x) \wedge S_2(y))) \\ &\wedge \forall x, y (\text{edge}(x, y) \rightarrow \neg (S_3(x) \wedge S_3(y)))) \end{aligned}$$

What does this do?

MSO as a logic for graphs

Example of MSO formula

$$\begin{aligned} \exists S_1 \exists S_2 \exists S_3 (&\forall x (S_1(x) \vee S_2(x) \vee S_3(x)) \\ &\wedge \forall x \neg (S_1(x) \wedge S_2(x)) \wedge \neg (S_1(x) \wedge S_3(x)) \wedge \neg (S_2(x) \wedge S_3(x)) \\ &\wedge \forall x, y (\text{edge}(x, y) \rightarrow \neg (S_1(x) \wedge S_1(y))) \\ &\wedge \forall x, y (\text{edge}(x, y) \rightarrow \neg (S_2(x) \wedge S_2(y))) \\ &\wedge \forall x, y (\text{edge}(x, y) \rightarrow \neg (S_3(x) \wedge S_3(y)))) \end{aligned}$$

What does this do? **Checks if graph is 3-colorable.**

MSO on graphs can express many classical NP-complete problems.

MSO as a logic for graphs

Graphs are a very general structure, how do we automata on them?

Wait...

MSO as a logic for graphs

Graphs are a very general structure, how do we automata on them?

Wait...

Let's just use tree automata anyway!

Let us see if we can generalize trees to something more expressive, and then use the tree automata result we already have.

How to build a tree?

- ◇ Start with a vertex.
- ◇ Repeatedly add vertices v adjacent to some previously existing vertex u .

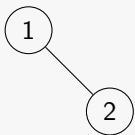
Tree



How to build a tree?

- ◇ Start with a vertex.
- ◇ Repeatedly add vertices v adjacent to some previously existing vertex u .

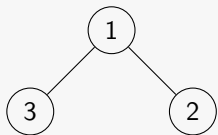
Tree



How to build a tree?

- ◇ Start with a vertex.
- ◇ Repeatedly add vertices v adjacent to some previously existing vertex u .

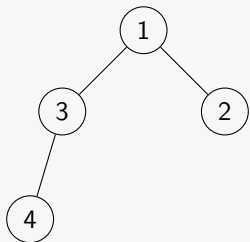
Tree



How to build a tree?

- ◇ Start with a vertex.
- ◇ Repeatedly add vertices v adjacent to some previously existing vertex u .

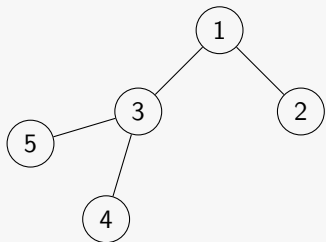
Tree



How to build a tree?

- ◇ Start with a vertex.
- ◇ Repeatedly add vertices v adjacent to some previously existing vertex u .

Tree



How do we generalize this process?

Make the tree "fatter": our new nodes ("bags") will now represent a small clique. We add new nodes while keeping the tree-of-cliques structure.

k-tree

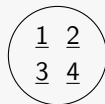
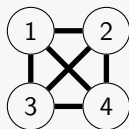
A *k*-tree is a graph that can be created by the following procedure:

- ◇ Start with a $(k + 1)$ -clique.
- ◇ Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k -clique (therefore forming another $(k + 1)$ -clique).

How do we generalize trees?

- ◇ Start with a $(k + 1)$ -clique.
- ◇ Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k -clique (therefore forming another $(k + 1)$ -clique).

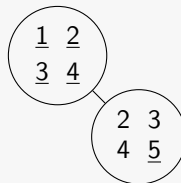
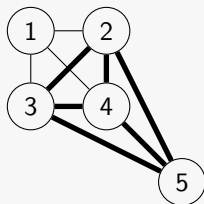
3-tree



How do we generalize trees?

- ◇ Start with a $(k + 1)$ -clique.
- ◇ Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k -clique (therefore forming another $(k + 1)$ -clique).

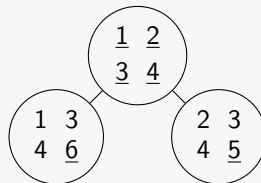
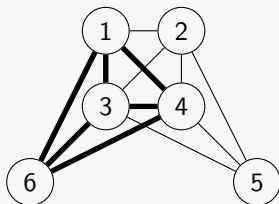
3-tree



How do we generalize trees?

- ◇ Start with a $(k + 1)$ -clique.
- ◇ Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k -clique (therefore forming another $(k + 1)$ -clique).

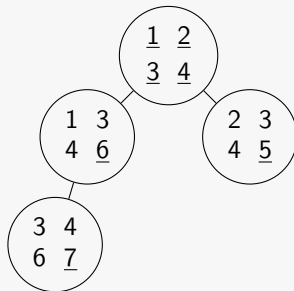
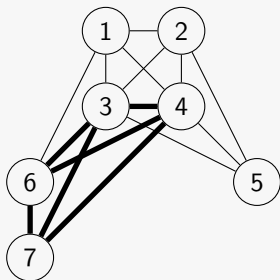
3-tree



How do we generalize trees?

- ◇ Start with a $(k + 1)$ -clique.
- ◇ Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k -clique (therefore forming another $(k + 1)$ -clique).

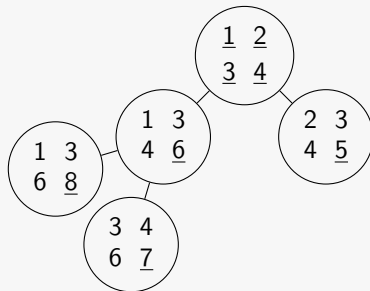
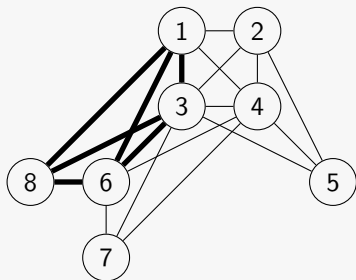
3-tree



How do we generalize trees?

- ◇ Start with a $(k + 1)$ -clique.
- ◇ Repeatedly add vertices v adjacent to U such that $|U| = k$ and U is a k -clique (therefore forming another $(k + 1)$ -clique).

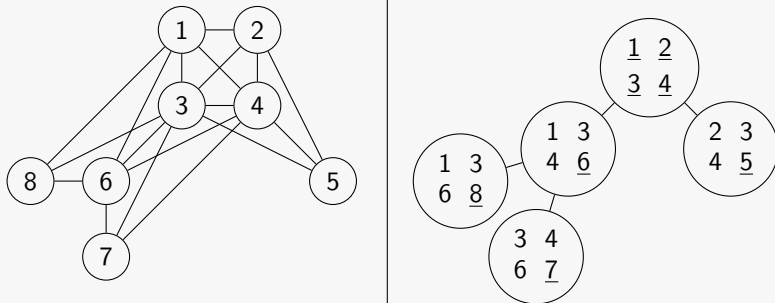
3-tree



How do we generalize trees?

- ◇ The graph G formed is a k -tree
- ◇ The respective tree of bags is a **tree decomposition of G of width k**

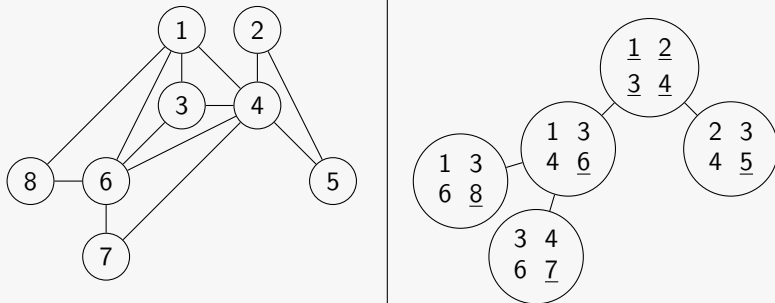
3-tree and tree decomposition of width 3



How do we generalize trees?

- ◇ A partial k -tree is a subgraph of a k -tree
- ◇ Graphs of treewidth at most k are the partial k -trees

Partial 3-tree and tree decomposition of width 3



Treewidth

This is not very helpful for automata design, let us look at what this decomposition actually means.

Characterizing partial k -trees

Tree decomposition

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

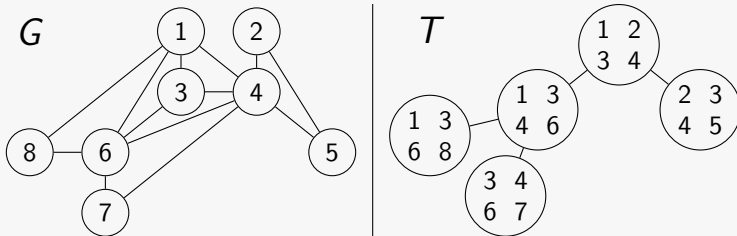
1. Every vertex is in some bag ($\forall v \in V, \exists X_i | v \in X_i$)
2. Every edge is in some bag ($\forall (u, v) \in E, \exists X_i | u, v \in X_i$)
3. Every vertex appears in a connected subgraph of T

Characterizing partial k -trees

Tree decomposition

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

1. Every vertex is in some bag ($\forall v \in V, \exists X_i | v \in X_i$)
2. Every edge is in some bag ($\forall (u, v) \in E, \exists X_i | u, v \in X_i$)
3. Every vertex appears in a connected subgraph of T

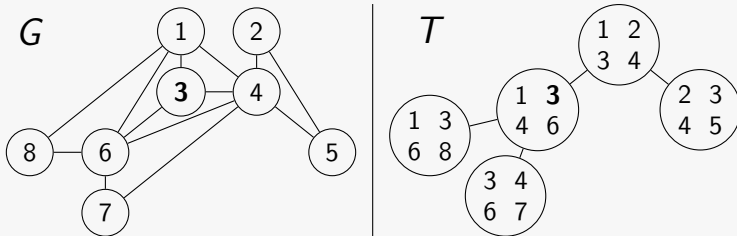


Characterizing partial k -trees

Tree decomposition

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

1. **Every vertex is in some bag** ($\forall v \in V, \exists X_i | v \in X_i$)
2. Every edge is in some bag ($\forall (u, v) \in E, \exists X_i | u, v \in X_i$)
3. Every vertex appears in a connected subgraph of T

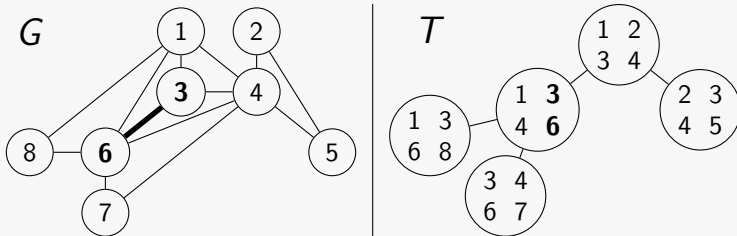


Characterizing partial k -trees

Tree decomposition

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

1. Every vertex is in some bag ($\forall v \in V, \exists X_i | v \in X_i$)
2. **Every edge is in some bag** ($\forall (u, v) \in E, \exists X_i | u, v \in X_i$)
3. Every vertex appears in a connected subgraph of T

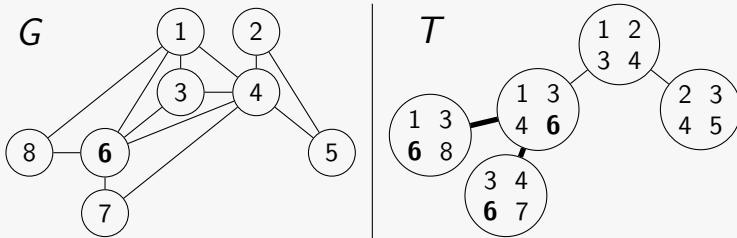


Characterizing partial k -trees

Tree decomposition

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

1. Every vertex is in some bag ($\forall v \in V, \exists X_i | v \in X_i$)
2. Every edge is in some bag ($\forall (u, v) \in E, \exists X_i | u, v \in X_i$)
3. **Every vertex appears in a connected subgraph of T**



Characterizing partial k -trees

Treewidth

A tree decomposition of a graph $G = (V, E)$ is a tree T of bags X_i such that

1. Every vertex is in some bag ($\forall v \in V, \exists X_i | v \in X_i$)
2. Every edge is in some bag ($\forall (u, v) \in E, \exists X_i | u, v \in X_i$)
3. Every vertex appears in a connected subgraph of T

The width of T is $\max_i |X_i| - 1$.

The treewidth of a graph is the lowest k such that G has a tree decomposition of width k .

We can use tree automata over the tree decomposition.

Why does each of these criteria matter?

How do we reason about partial k -trees

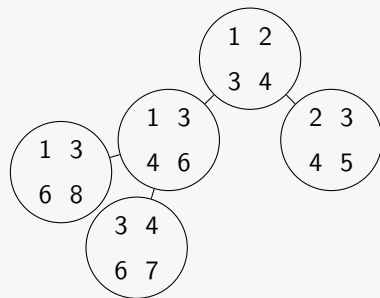
Tree automata

We use tree automata over the tree decomposition.

Each bag contains a constant amount of vertices, so it is easy to reason about properties within a bag.

Properties of a tree decomposition guarantee that keeping a local state for each bag is enough.

Tree decomposition



Courcelle's theorem

Every graph property definable in the monadic second-order logic of graphs can be decided in linear time on graphs of bounded treewidth.

The idea is straightforward (but complicated in practice):

- ◇ Find the tree decomposition of the graph at hand
- ◇ Construct a tree automata that runs on tree decomposition of the graph and checks for the desired property

This is all linear if the graph has constant treewidth.

Can we get even more expressive?

No

Seese

If a class of graphs has a decidable MSO theory, that class has bounded treewidth.

So, sadly, we must stop here.

1. MSO over Finite Words
2. MSO over Infinite Words
3. MSO over Trees
4. MSO over Graphs
- 5. Conclusion**

Decidability of MSO

MSO model checking

Finite words	decidable (BET)
Infinite words	decidable (Büchi's Theorem)
Finite trees	decidable
Infinite trees	decidable (Rabin)
Bounded-treewidth graphs	decidable (Courcelle)
Other graphs	undecidable (Seese)

Model checking is non-elementary in the formula size but admits linear testing.

For that amount of decidability, this is as expressive as it gets.